

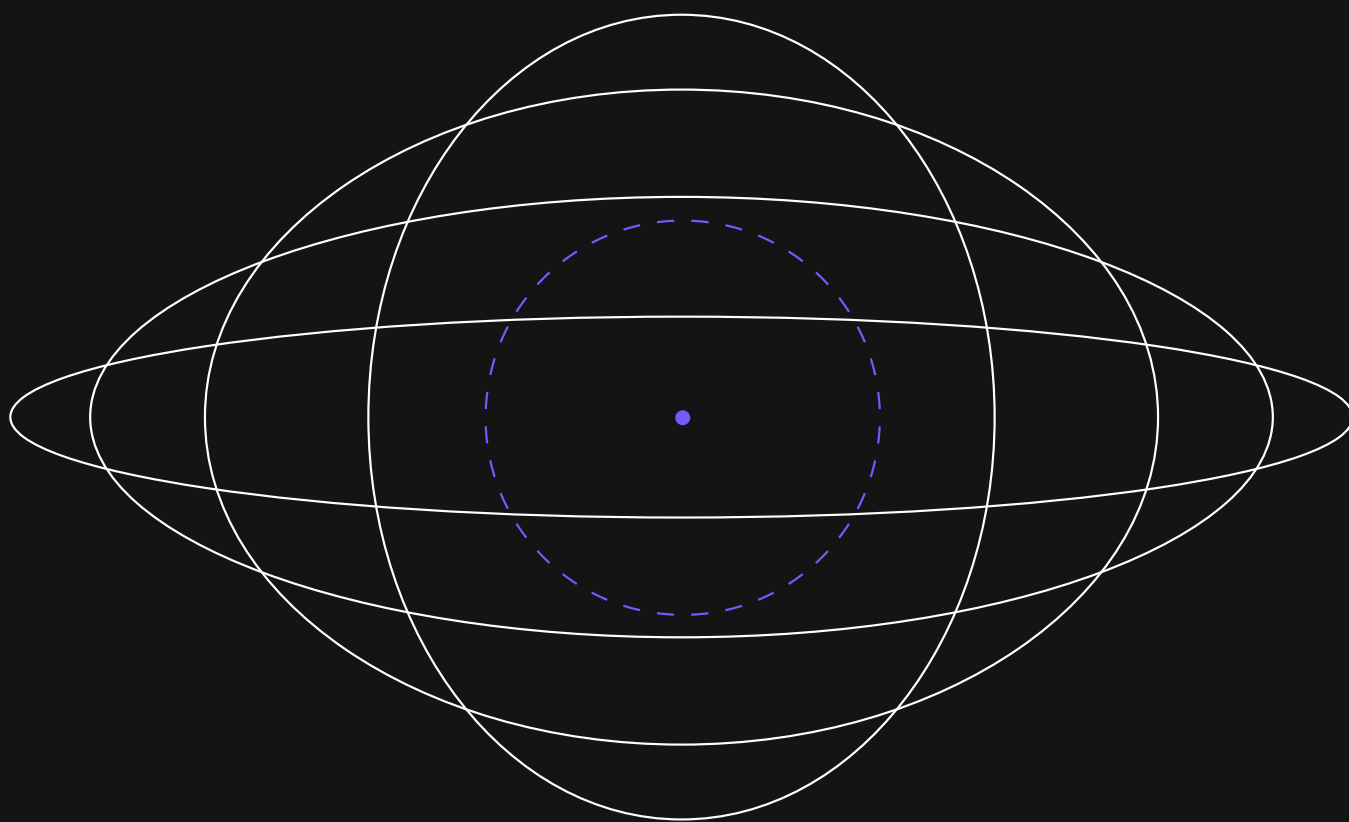


Машинное обучение

Лонгрид

01

ML Lifecycle. Введение



Содержание

1. Процесс обучения ML-моделей	3
1.1. CRISP-DM: цикл работы над ML-задачей	4
1.2. Уточнение и постановка задачи (Problem Definition)	6
1.3. Сбор, подготовка и анализ данных (Data Mining)	7
1.4. Выбор алгоритма, признаков и способа контроля модели	10
1.5. Обучение модели (Fit)	12
1.6. Предсказания и оценка качества (Predict & Evaluation)	13
1.7. Deploy / Release / Online Evaluation / Debug / Monitoring / Maintenance	14
1.8. Отчётность, презентация, коммуникация, документация	15
2. Основные термины ML	16
2.1. Основные задачи машинного обучения	16
2.2. Постановка ML-задачи	17
2.3. Постановка задачи регрессии	18
2.4. Постановка задачи классификации	20
2.5. Постановка задачи кластеризации	22
3. Если кратко	24

ML Lifecycle. Введение

Контекст	На первой неделе ты познакомишься с темами, которые пригодятся в следующих материалах курса. Разберём основные термины машинного обучения, этапы жизненного цикла ML - от постановки задачи до презентации результатов, а также метрики и функционалы потерь в задачах регрессии.
Применение	Изучив эти темы, ты будешь увереннее разбираться в ключевых понятиях ML, этапах работы над проектом и типичных метриках в задачах регрессии. Это база, которая необходима для будущих специалистов по машинному обучению, аналитиков, инженеров и Data Scientists.

Сегодня мы начинаем знакомство с машинным обучением. За этим термином скрываются очень разные задачи: предсказать стоимость квартиры, найти мошенническую операцию, распределить обращения клиентов или собрать персональные рекомендации. На курсе мы постепенно разберёмся, какие модели решают такие задачи и как они устроены.

В первом лонгриде посмотрим на весь процесс работы над ML-проектом. Прежде чем обучать модель, нужно сформулировать задачу, разобраться с данными и выбрать способ оценки. После обучения работу тоже нельзя считать законченной: решение нужно проверить, запустить и дальше следить за его качеством. Мы пройдем весь этот путь, а затем познакомимся с основными терминами и типами задач машинного обучения, чтобы дальше говорить на одном языке.

1. Процесс обучения ML-моделей

ML-решения могут сильно различаться по устройству: где-то достаточно одной классической модели, а где-то нужен сложный DL-пайплайн с несколькими сервисами, регулярным переобучением и мониторингом. При этом общий путь проекта сохраняется - от постановки задачи до запуска и поддержки готового решения. Этот путь называют жизненным циклом машинного обучения.

ОПРЕДЕЛЕНИЕ

Жизненный цикл машинного обучения (англ. ML lifecycle) — это последовательность этапов, через которые проходит ML-проект: от постановки задачи до выпуска в production и регулярного обновления.

1.1. CRISP-DM: ЦИКЛ РАБОТЫ НАД ML-ЗАДАЧЕЙ

ML Lifecycle можно представить через схему CRISP-DM (Cross-Industry Standard Process for Data Mining).

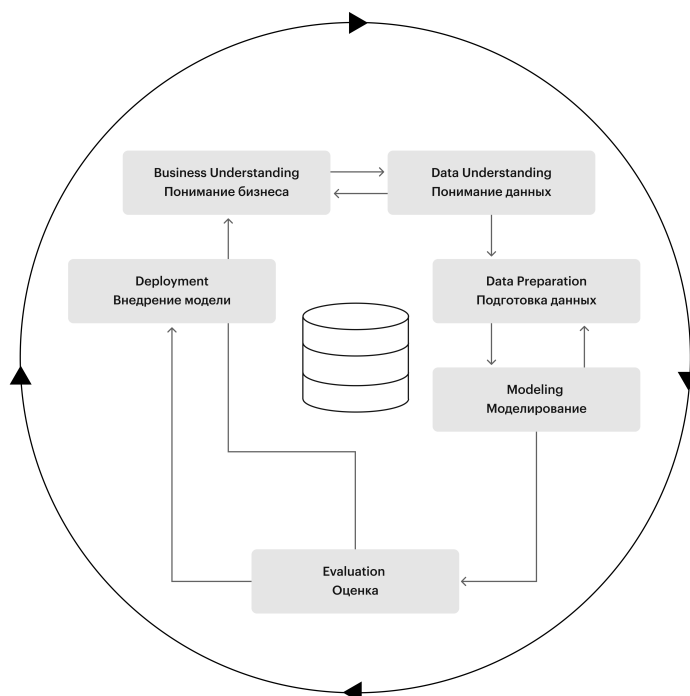


Рисунок 1. Схема CRISP-DM

ОПРЕДЕЛЕНИЕ

CRISP-DM — это классический фреймворк с практическими рекомендациями по структуре работы над проектами в сфере анализа данных и машинного обучения.

CRISP-DM показывает, что жизненный цикл не линейный, а итерационный и состоит из шести основных этапов. На каждом этапе разработчики отвечают на вопросы:

1. Business Understanding (понимание бизнеса)

- Что именно нужно сделать?
- Как это связано с бизнесом? Как это помогает бизнесу достичь целей?
- Как измерить успех?

2. Data Understanding (понимание данных)

- Какие данные у нас есть? Этого достаточно или нужно привлекать дополнительные источники?
- Какая у данных структура? С какими типами данных мы работаем? Как реализована разметка данных?
- Что именно мы можем извлечь из данных, не привлекая дополнительные ресурсы?

3. Data Preparation (подготовка данных)

- Как чистить и преобразовывать данные? Какие признаки (и как) отобрать?
- Какие новые признаки можно создать из имеющихся?

4. Modeling (моделирование)

- Как подобрать и обучить модель?

5. Evaluation (оценка)

- Как оценить готовое решение?
- Что считать критерием успеха с технической точки зрения? Как это коррелирует с бизнес-требованиями?

6. Deployment (развёртка / внедрение модели в production)

- Как выпустить модель в production?
- Как отслеживать работу модели? Что делать, если модель работает хуже, чем ожидалось?
- Как наладить систему обновления модели?

На каждом этапе можно (и нужно) возвращаться назад. Например, при Evaluation может оказаться, что нужны новые признаки, а значит, нужно вернуться на этап Data Preparation. Не стоит воспринимать ML Lifecycle как конвейер — это всегда цикл, причём итеративный.

CRISP-DM — классика из Data Mining эпохи, которая помогает смотреть на задачи шире и мыслить на уровне процессов. Но CRISP-DM можно расширить, углубить и преобразовать в современный ML Lifecycle. Именно его мы будем изучать на этом курсе.



Внутри лонгридов есть подборка ключевых вопросов для ML-собеседований. Готовься к стажировке с помощью этого must-have контента по тегу «Interview Kit».

ML Lifecycle можно разделить на несколько этапов. В рамках нашего курса затронем каждый из них (см. рисунок 2).

Современный ML Lifecycle

Схема решения задачи

01 Уточнение и постановка задачи (Problem Definition)

понимание бизнес-проблемы, математическая формализация

02 Сбор, подготовка и анализ данных (Data Mining)

- понимание исходных данных
- сбор данных (Data collection)
- предобработка данных (Data cleaning) / подготовка данных для модели
- разведочный анализ (Exploratory Data Analysis)

Машинное обучение (Machine Learning)

03 Выбор

Алгоритма

- модели (Model selection)
- способа обучения: гиперпараметры, методы оптимизации

Контроля

- функции ошибки (Metric selection)
- способа контроля (разбиение train/test/valid)

Признаков

- генерация (Data coding = feature engineering)
- селекция (feature selection)

04 Обучение (fit)

05 Предсказание (predict) и проверка качества (evaluation)

(может ещё постобработка Post-processing)

06 Deploy / Release / Online evaluation / Debug / Monitoring / Maintenance

переобучаем алгоритм на всех данных

07 Отчётность, презентация, коммуникация

визуализация данных, модели, результатов

Рисунок 2. Семь этапов ML Lifecycle

Основной акцент сделаем на блоках 3, 4 и 5 — выборе, обучении и оценке качества моделей. На семинарах и при решении домашних заданий понадобятся блок 1 (уточнение и постановка задач) и блок 2 (сбор, подготовка и анализ данных). Работа над этими этапами решения ML-задачи требует насмотренности и опыта, которые приходят с практикой. Блок 6 затронем вскользь, так как будут отдельные курсы про релиз моделей в production. Блок 7 мы будем изучать по ходу курса, но важно понимать, что навыки коммуникации и презентации приходят с опытом. Поэтому в рамках курса ты будешь решать прикладные примеры и тренировать навыки презентации во время защиты проекта по итогам курса.

Разберём каждый блок схемы по решению задач.

1.2. УТОЧНЕНИЕ И ПОСТАНОВКА ЗАДАЧИ (PROBLEM DEFINITION)

В компаниях ML-отделы не всегда выступают инициаторами проектов. Гораздо чаще от бизнеса приходит запрос на решение реальной проблемы. Компания хочет что-то улучшить, оптимизировать или достичь своих целей, но не всегда понимает, как это сделать.

В таких ситуациях боль бизнеса часто хотят решить с помощью ML, хотя на деле достаточно бейзлайна (простого решения без сложных моделей).

В этой ситуации ML-специалист разбирается в сути проблемы и предлагает оптимальный способ её решения. Нужно понять:

- В чём именно проблема бизнеса? Как это выражается в бизнес-метриках?
- Какой идеальный результат ожидает бизнес? Как бизнес предполагает его использовать на практике?
- Какие есть ограничения, например по доступным вычислительным ресурсам или скорости работы решения? Есть ли какие-то дополнительные задачи?
- Как бизнес будет измерять успех?

Этап 1 — это трансформация боли бизнеса в IT/ML-задачу. Например, задача «хотим уменьшить отток клиентов» превращается в «строим классификатор, предсказывающий вероятность ухода клиента». Problem Definition позволяет заранее согласовать ключевые позиции, определить критерии успеха и ограничения. На этом этапе нужно задать вопрос: точно ли здесь нужен ML, или можно решить проблему без него?

Результат Problem Definition

- Поняли, какую бизнес-проблему решаем и точно ли нужно использовать ML.
- Выяснили, какую бизнес-метрику оптимизируем (не RMSE ради RMSE, а, например, выручку/вовлечённость/NPS).
- Связали техническую метрику с бизнес-реалиями.
- Формализовали задачу: классификация / регрессия / ранжирование / что-то другое.
- Уточнили ограничения и пожелания: время обучения, время инференса (ответа модели), технические ограничения, требования к интерпретируемости.
- Продумали, как будем использовать решение на практике и оценивать его успех: офлайн-метрики, A/B-тест, бизнес-индикаторы.

1.3. СБОР, ПОДГОТОВКА И АНАЛИЗ ДАННЫХ (DATA MINING)

Фундамент любого ML-решения — это данные: датасеты, разбитые на тренировочные, валидационные и тестовые выборки.

Этот этап разработки занимает больше всего времени и самый трудозатратный. Проблемы могут возникнуть на каждом шагу. Например, неполные или неточные данные, плохо размеченные или вовсе не доступные в проде. Если обучающие данные будут некачественными, то команда может столкнуться с трудностями.

Возможные сложности с данными: примеры проблем, которые на этом этапе можно найти и нужно решить

- **Дисбаланс классов**

Для некоторых задач, например кредитного скоринга или диагностики заболеваний, дисбаланс классов отражает реальное распределение событий. Его нужно учитывать при подготовке данных, обучении и оценке модели.

- **Шум**

Шум — это случайные ошибки и искажения в данных. Он может появляться при сборе, измерении, передаче или разметке данных. Такие данные ухудшают обобщающую способность модели, а борьба с ними требует контроля качества, реализовать который не всегда возможно.

Примеры шума:

- неверная разметка,
- дублирующаяся разметка,
- различные артефакты и аномалии,
- сильные различия в качестве и разрешении изображений внутри одной выборки.

- **Пропущенные значения**

Пропущенные значения возникают, когда для части объектов отсутствуют значения отдельных признаков или таргета. Причины пропусков важно изучить: иногда сам факт пропуска содержит полезную информацию.

Степень влияния качества данных на итоговую метрику различается для разных AI-задач. Но грамотная разметка улучшает результаты даже на бенчмарк-датасетах. Например, [по ссылке](#) можно послушать про эксперимент по улучшению разметки PASCAL VOC 2012, в результате которого mAP конечной модели увеличился на 13%.

Понимание и ревизия данных Возможные проблемы с данными известны, теперь их нужно проанализировать:

- Какие источники данных мы используем? Насколько они надёжны и актуальны?
- Достаточно ли у нас данных для обучения модели или нужно дополнительно собирать/генерировать данные?
- Соответствуют ли собранные/найденные/полученные данные реальной задаче? Нет ли смещения (Bias), которое может повлиять на результаты?
- Как мы будем обрабатывать персональные и чувствительные данные, чтобы соответствовать требованиям [GDPR](#), [HIPAA](#) и других регуляций (если это необходимо в нашем проекте)?
- Какие таблицы и поля доступны? Как они формируются и обновляются?
- Какие признаки будут реально доступны в момент предсказания?

Важно не просто «посмотреть на табличку с данными», а понять происхождение, логику, контекст и оценить потенциальные риски: утечки, нестабильные источники, циклические зависимости.

Сбор данных

Если нужного датасета нет (а для большинства реальных задач это почти всегда так), его нужно собрать. В крупных компаниях разработчики помогают с этой задачей, но редко отвечают за неё напрямую. При этом они должны убедиться, что собранные данные им подходят.

Если же приходится собирать данные самостоятельно, то для этого можно использовать SQL-выгрузки, API, логи и внешние источники. При этом надо постоянно проверять — можно ли будет стабильно получать эти данные в production-среде.



Распространённая ошибка — собрать «идеальные» данные, которые невозможно использовать в реальной системе. Именно поэтому нужно проверять, будет ли доступ к каждому признаку в реальной среде.

Отдельно отметим процесс разметки данных. Если её нет или она вызывает вопросы, разработчикам (при помощи других отделов) придётся ответить на ряд дополнительных вопросов:

- Требуется ли разметка данных для обучения модели? Если да, то какой уровень детализации необходим?
- Какими методами мы будем проводить разметку (ручная разметка, полуавтоматическая, автоматическая)?
- Кто будет заниматься разметкой данных — внутренняя команда, аутсорс или краудсорсинговая платформа вроде Toloka?
- Как мы будем проверять качество разметки и минимизировать ошибки аннотаторов (специалистов, которые размечают данные)?

Очистка и подготовка Когда уже есть размеченные данные, их нужно очистить и подготовить для дальнейшего анализа и подачи в модель:

- Обработать пропуски (при этом задуматься, что они значат или могут значить в контексте решения).
- Убрать или трансформировать выбросы.
- Поработать с шумом: дубли, бессмысленные значения, невалидные строки.

Задача этого этапа не сводится к «удалению всего подозрительного». Здесь важно провести глубокую смысловую работу с контекстом подозрительных данных. Иногда пропуск — это не ошибка, а важный сигнал («Клиент не заполнил анкету — может, потому что передумал?»). Уже на этом этапе команда разработки может представить важные для бизнеса выводы.

Разведочный анализ данных (EDA)

ОПРЕДЕЛЕНИЕ

Exploratory Data Analysis (EDA) — это финальный шаг в Data Mining этапе.



Важно. EDA это не про графики ради красоты!

EDA позволяет:

- понять, какие признаки влияют на целевую переменную и как они связаны между собой;
- найти смещения, перекосы, подозрительные и интересные зависимости;
- оценить баланс классов, распределение признаков, наличие или отсутствие корреляций;
- отсеять лишние признаки и придумать, как из существующих сделать новые.

Хороший EDA помогает заметить проблемы и особенности данных до обучения модели — и использовать эти инсайты для принятия важных бизнес-решений.

ПРИМЕР

«80% ушедших клиентов звонили в поддержку за 3 дня до ухода» — это понятный, значимый инсайт, который бизнес может сразу использовать без всяких ML-моделей.

Результат Data Mining



Важно. Качество ML-модели определяется качеством данных, на которых эта модель обучена. Здесь работает принцип «garbage in — garbage out» («мусор на входе — мусор на выходе»). Если на входе шум, баги и нестыковки, то никакой ML не поможет — на выходе всё равно получится мусор, только в красивой обёртке.

- Проверили, достаточно ли данных для решения задачи (объём, качество, доступность в проде).
- Учили особенности: дисбаланс классов, пропуски, выбросы, шум — и знаем, как с ними работать.
- Провели ревизию данных: поняли смысл признаков, проверили валидность, зафиксировали потенциальные проблемы.
- Если данных нет — знаем, как и где их собирать (базы данных, API, логи, внешние источники).
- Продумали процесс разметки (если нужна): кто и как делает разметку, чем проверяем качество.
- Провели очистку и подготовку данных: пропуски, выбросы, шум — не просто убрали, а осмыслили.
- Сделали EDA: поняли структуру данных, выявили зависимости, построили первичные гипотезы.
- Подготовили датасет для модели и список гипотез/наблюдений для дальнейшей работы.

1.4. ВЫБОР АЛГОРИТМА, ПРИЗНАКОВ И СПОСОБА КОНТРОЛЯ МОДЕЛИ

Этот этап ML Lifecycle — один из ключевых для этого курса. В следующих неделях мы будем подробно говорить о выборе моделей, различных подходах к их оценке, преимуществах и недостатках разных алгоритмов. Это позволит научиться выбирать подход к разработке именно для вашей задачи.

Прежде чем учить какую-то модель, разработчикам нужно знать:

- Какую модель использовать? Почему именно её?
- Какие признаки будут доступны? Почему именно они?
- Как будет проводиться оценка качества? Почему именно так?

На данном этапе ML Lifecycle, когда уже есть данные, с которыми можно работать, выбираем конкретные инструменты решения. Этот процесс будет строиться на итеративных экспериментах, которые помогут подобрать наилучшую модель, самые релевантные признаки и подготовить алгоритм оценки результата.

Выбор алгоритма (Model Selection) В рамках этого шага разработчики определяют, какую модель они хотят использовать для решения задачи. Здесь важно подобрать инструмент для конкретной задачи и имеющихся ограничений.

Вот несколько советов для выбора алгоритма:

- Выбирай семейство моделей и конкретную модель с учётом требований к скорости, интерпретируемости и объёму данных.
Определи, сколько времени и какое количество машинных ресурсов есть для обучения и инференса (процесса применения) модели. Насколько предсказания модели должны быть интерпретируемы, какие есть ограничения и пожелания.



В последние годы чаще всего используют бустинговые модели, поэтому в этом курсе на них мы сделаем большой акцент. Но и простые модели могут работать не хуже, чем сложные, поэтому с ними ты тоже познакомишься. Многие из них — базис для более глубоких подходов.

- Заранее продумай, как будешь обучать модель и подбирать её гиперпараметры.
Например, уточни, что требуется от модели в будущем. Планируется ли обновление решений? Должна ли модель учиться «на лету»?

Выбор контроля (Metric Selection) Обученную модель нужно оценить по всем правилам, чтобы не получить в итоге красивое, но бесполезное решение. Нужно заранее продумать:

- функцию ошибки: что конкретно оптимизировать при обучении модели;
- метрику качества: на какой показатель ориентироваться при валидации модели;
- способ валидации: как именно данные будут разделены при оценке качества модели — Train/Validation/Test, k-Fold, Time Series Split для временных рядов и так далее.

Эти аспекты мы также разберём в данном курсе.

Работа с признаками (Feature Engineering) Качество модели зависит от данных и их признаков не меньше, чем от выбора алгоритма. Имеющиеся признаки — всё, что модель знает об окружающем мире. В работе с признаками используем:

1. Генерацию. Из исходных данных выделяем новые смыслы, которые используем для агрегатов, контекстных фичей и трансформаций, преобразования текстовых/категориальных признаков.
 - Например, частый пример генерации признаков — извлечение временных признаков из даты и времени. Если есть дата-время события: 2024-06-27 14:45:00 — можно на его основе сгенерировать новые признаки:
 - `day_of_week` -> четверг (4);
 - `hour` -> 14;
 - `is_weekend` -> False;
 - `month` -> 6;
 - `part_of_day` -> 'afternoon'.
 - Эти признаки помогут модели уловить дополнительные закономерности, например:
 - пользователи чаще совершают покупки вечером или в выходные;
 - определённые дни недели связаны с ростом/падением спроса.
2. Селекцию. Можно удалить лишнее, уменьшив таким образом шум — избыточные или потенциально «грязные» признаки часто хуже влияют на качество модели, чем их отсутствие.



Важно. Следи, чтобы при генерации и отборе признаков не возникла Data Leakage. Утечка данных происходит, когда при обучении модель получает информацию, которая не будет доступна в момент реального предсказания: например, данные из будущего, сведения о таргете или статистики, рассчитанные с использованием валидационной и тестовой выборок. Из-за этого оценка качества модели оказывается завышенной.

Результаты Model и Metric Selection и Feature Engineering

- Выбран алгоритм (Model Selection):
 - Понимаем, какую модель используем и почему (учли ограничения по времени обучения, инференса, интерпретируемости, объёму данных).
 - Выбран подходящий инструмент: часто это бустинг, но в ряде случаев — простые модели.
- Определён способ оценки качества (Metric Selection):
 - Чётко понимаем, какую метрику и Loss используем: что оптимизируем при обучении, что измеряем при валидации.
 - Продумали способ валидации: Train/Test Split, k-Fold, Time Series Split — в зависимости от задачи.
- Подготовлен набор признаков (Feature Engineering):
 - Сгенерировали новые признаки, если нужно (например, из даты: день недели, час, `is_weekend`).
 - Отобрали релевантные фичи, убрали лишнее, минимизировали шум.
 - Проверили, нет ли Data Leakage (модель не должна «подглядывать» в будущее).
- Есть план по обучению модели:
 - Понимаем, как будет идти обучение, подбор гиперпараметров, сравнение моделей.
 - Знаем, какие эксперименты будем проводить и какие вопросы они должны закрыть.

1.5. ОБУЧЕНИЕ МОДЕЛИ (FIT)

Всё то, что происходило на предыдущих этапах ML Lifecycle, вело к обучению модели. Именно здесь данные, признаки и выбранная модель впервые встречаются на практике.

Но этот этап не просто механический Fit в коде. Важно помнить о связях всех элементов, вдумчиво анализировать результат и при необходимости адаптировать, возвращаясь к предыдущим этапам для доработки или оптимизации.

Для успеха на этом этапе нужно:

- Следовать выбранной методологии, использовать только «правильные» фичи (которые действительно можно стабильно получать в production-среде).
- Применять подходящий способ обучения — например, учить модель, регулируя её гиперпараметры и отслеживая, как ведёт себя метрика на валидации.
- Сравнивать разные модели или подходы — иногда лучший результат показывает простой алгоритм. При этом сравнение должно быть всесторонним. Важно помнить не только о метриках качества, но и о возможных ограничениях времени обучения и инференса, затрачиваемых ресурсов и так далее.
- Следим за поведением модели во время обучения. Резкий необъяснимый рост качества может указывать на Data Leakage, а заметное ухудшение качества на новых данных — на переобучение.



Совет. Не бойся возвращаться к предыдущим этапам. Работа над проектом редко бывает линейной (последовательность 1-2-3-4-5 — недостижимый идеал), почти всегда приходится что-то доработать в данных, логике валидации, списке признаков, а иногда — и в постановке задачи.

То есть Model и Metric Selection и Feature Engineering технически и логически связаны с Fit, давая разработчикам возможность экспериментировать и достигать желаемого результата.

Результат Fit

- Модель обучена на данных.
- Используем только те признаки, которые реально доступны в проде, чтобы не было странных моментов на тестах.
- Обучение идёт с учётом валидации и контроля переобучения:
 - Следим за поведением метрик на обучении/валидации.
 - Проверяем, не случилась ли Data Leakage.
 - Отслеживаем переобучение по ухудшению качества на новых данных.
- Сравниваем разные модели/подходы: иногда простая модель работает лучше сложной, важно это проверить.
- Итеративно дорабатываем пайплайн. Если нужно, возвращаемся к признакам, алгоритму, настройкам.

1.6. ПРЕДСКАЗАНИЯ И ОЦЕНКА КАЧЕСТВА (PREDICT & EVALUATION)

Обучение пройдено. Несколько шагов назад уже был сделан выбор, как именно будет проводиться валидация. Пора воплотить это в жизнь.

Ключевой момент — грамотно выстроенный процесс:

- Предсказания делаем только на новых данных. Используем честную отложенную выборку или реализуем другой способ валидации, например k-Fold.
- Расчёт производим по заранее определённым ML-метрикам, внимательно анализируем результат. Обязательно стоит сравнить качество полученной модели с бейзлайном.
 - Отдельно можно попробовать перевести ML-метрики в плоскость бизнес-метрик/показателей/индикаторов, то есть связать бизнес-значимость с увеличением точности модели.
- Оцениваем модель на предмет устойчивости и стабильности — важно узнать (особенно на небольших выборках), не «повезло» ли нам?
- Проверяем, нужна ли предсказаниям постобработка, например: сглаживание, объединение, калибровка, кастомные правила.
- Сопоставляем ограничения и пожелания бизнеса с получившимся результатом.



Помни: Оценка на новых данных показывает, насколько хорошо модель работает с объектами, которых не видела при обучении. Тестовую выборку не используют при обучении и подборе модели. Кроме того, реальную пользу решения оценивают после внедрения с помощью онлайн- и бизнес-метрик.

Результат Predict & Evaluation

- Провели честную оценку модели: предсказания делаем только на новых данных (отложенная выборка или k-Fold).
- Считаем метрики строго по заранее выбранному плану (не подстраиваем метрики под результат постфактум).
- Сравнили с бейзлайном: новая модель действительно лучше?
- Связали ML-метрики с бизнес-метриками: понимаем, как увеличение точности влияет на деньги/процесс.
- Оценили устойчивость модели: нет ли случайных всплесков качества из-за удачного разбиения данных.
- Подумали, нужна ли постобработка предсказаний: сглаживание, калибровка, кастомные правила.
- Проверили, совпадает ли результат с ограничениями и планами, зафиксированными на этапе постановки задачи.

1.7. DEPLOY / RELEASE / ONLINE EVALUATION / DEBUG / MONITORING / MAINTENANCE

Готовое решение должно приносить ценность бизнесу. Для этого разработчики интегрируют решение в production-среду — реальные IT- и бизнес-процессы.

Именно на этом шаге исследовательский код превращается в реальный продукт. Как это происходит:

- Деплой и запуск: модель «переезжает» из ноутбука или тестовой среды в production. Часто для этого нужны обёртки, REST API, пайплайны или специальные ML-сервисы. Не всегда эту задачу выполняют Data Scientists или ML Engineers, во многих компаниях этим занимаются production/inference-подразделения.
- Переобучение на всех данных. Перед запуском модель при необходимости переобучают на всех данных, выделенных для обучения, с уже выбранными настройками.
- Мониторинг и отладка: после релиза разработчики следят за ключевыми показателями модели (ML-метрики, скорость инференса, устойчивость), оперативно реагируя на баги и сбои.
- Поддержка и регулярное переобучение. Данные меняются и модель устаревает — к этому должны быть готовы и разработчики, и бизнес. Грамотно выстроенная система логирования и мониторинга перформанса модели позволит не упускать из виду её качество и разработать план по её поддержке — будь то плановой или реактивной в случае поступления новых вводных.



Совет. Не забывай про систему логов, оповещений на подозрительное поведение и документацию кода. Хорошо развёрнутая модель — та, которая не превращается в «ещё одну чёрную коробку».

Результат Deploy У компании есть модель, развёрнутая в production-среде, которая готова принимать и обрабатывать новые данные.



В рамках данного курса мы не будем подробно останавливаться на этом этапе жизненного цикла. На эту тему есть отдельные курсы по MLOps.

- Модель готова к продакшену: перевели из ноутбука в реальную систему (через API, пайплайны, ML-сервисы).
- При необходимости переобучили модель на всех данных, выделенных для обучения, с уже выбранными настройками.
- Настроен мониторинг: следим за качеством предсказаний, скоростью, стабильностью работы.
- Готовы к отладке и поддержке: предусмотрены логи, алёрты на подозрительное поведение.
- Понимаем, что модель устаревает — есть план регулярного переобучения или обновления.
- Учли инфраструктуру: договорились, кто за что отвечает при работе модели.

1.8. ОТЧЁТНОСТЬ, ПРЕЗЕНТАЦИЯ, КОММУНИКАЦИЯ, ДОКУМЕНТАЦИЯ

Результаты недостаточно просто получить. На любом из этапов может появиться необходимость представить результаты — интерпретировать, визуализировать и донести на понятном языке до команды разработки и бизнеса.

Презентация результатов — экзамен. Недостаточно просто показать результат, важно аргументировать его ценность или объяснить, почему отсутствие результата — тоже результат.

На этом этапе пригодятся:

- Визуализация: понятные подробные графики, дашборды, инфографика, чтобы увидеть не только «что вышло», но и «почему оно сработало» (или не сработало).
- Документация: оформленное текстовое сопровождение работы — что делалось, какие гипотезы проверялись, какие решения принимались в процессе.
- Коммуникация: требуется представить результаты так, чтобы и разработчики, и бизнес-заказчики одинаково поняли выгоды и ограничения решения.
- Обратная связь: стоит обсудить с заказчиками и командой разработки, устраивает ли всех полученный результат. Выяснить, нужны ли доработки или итерации. ML-проекту редко удаётся долго функционировать без правок.



Навык презентации — важное умение для любого разработчика. Если построить хорошую модель и показать её реальную пользу, риски и дальнейшие шаги, можно стать для бизнеса не просто исполнителем, а партнёром в будущих проектах.

Результат

После этого этапа бизнес и разработчики понимают ценность решения, его ограничения, дальнейшие планы разработки, поддержки и оптимизации.

2. Основные термины ML

Теперь познакомимся с основными терминами машинного обучения. Сначала разберёмся, чем обучение с учителем отличается от обучения без учителя, а затем посмотрим, какие задачи относятся к каждому типу.

2.1. ОСНОВНЫЕ ЗАДАЧИ МАШИННОГО ОБУЧЕНИЯ

Есть две основные группы классических ML-задач:

- **Обучение с учителем** — тип машинного обучения, при котором модель обучается на примерах с известными правильными ответами — метками или таргетами.
 - Пример: обучаем модель определять, спам ли письмо. В датасете каждый email уже помечен как «спам» или «не спам».
 - Цель: научить модель предсказывать правильный ответ на новых, ещё не размеченных данных.
- **Обучение без учителя** — тип машинного обучения, при котором модель работает с неразмеченными данными: у неё нет информации, какой ответ правильный. Она должна сама выявить структуру или скрытые закономерности в данных.
 - Пример: анализируем поведение клиентов, чтобы сгруппировать их по схожести — без заранее заданных категорий.
 - Цель: найти скрытые структуры, шаблоны или зависимости в данных без использования меток.

Есть ещё обучение с подкреплением (Reinforcement Learning), но оно выходит за рамки нашего курса.

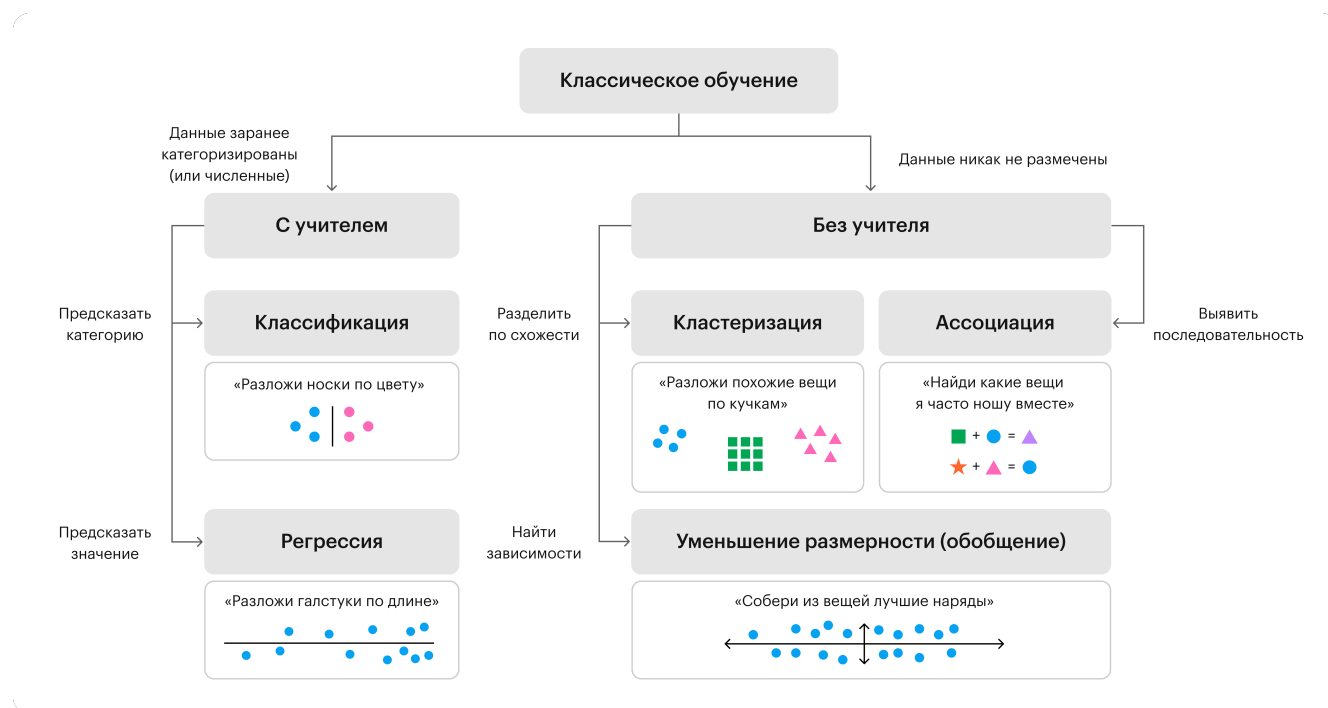


Рисунок 4. Разновидности классического обучения

Классические ML-задачи обучения с учителем:

- **Регрессия** — модели необходимо предсказать непрерывное число (например, цену квартиры).
- **Классификация** — модели необходимо выбрать один из классов из конечного множества вариантов (например, «кошка», «собака», «птица»).
- **Ранжирование** — модели необходимо упорядочить объекты по убыванию релевантности (например, персонализированные рекомендации для онлайн-магазина).

Классические ML-задачи обучения без учителя:

- **Кластеризация** — модели необходимо объединить объекты в группы (кластеры) на основе их внутреннего сходства без использования внешних меток / правильных ответов (например, сегментация клиентов онлайн-магазина).
- **Методы понижения размерности** — модели необходимо преобразовать данные из пространства с большим числом признаков в пространство с меньшим числом признаков, сохранив при этом наиболее важную информацию.
- **Детекция аномалий** — модели необходимо выявить объекты, которые существенно отличаются от «нормы» (например, определение мошеннических транзакций в банковском секторе).

При работе над проектом важно выбрать верную ML-задачу, ведь от неё будет зависеть формат целевой переменной, варианты моделей и возможности по оценке качества.

2.2. ПОСТАНОВКА ML-ЗАДАЧИ

Любой проект машинного обучения начинается с постановки задачи. Она отвечает на вопрос, что именно должна делать модель — предсказывать, классифицировать, детектировать, сегментировать, рекомендовать. Чётко определённая задача задаёт направление всей работе.

ОПРЕДЕЛЕНИЕ

Задача машинного обучения — это формальная постановка, в которой определены исходные данные, ожидаемый результат и способ оценки качества. По данным требуется построить алгоритм, который сможет предсказывать ответы для новых объектов или выявлять структуру данных.

Для постановки ML-задач нам понадобятся три базовых понятия. Множество объектов X есть в любой задаче. В задачах обучения с учителем также появляются известные ответы Y и целевая зависимость между объектами и ответами. В задачах без учителя алгоритм работает только с объектами X и ищет в них внутреннюю структуру.

1. **Множество объектов (X)** — входные данные (например, строки в таблице Excel). Каждый объект описывается с помощью признакового описания — набора характеристик. Зафиксируем, что каждый объект описывается n признаками. Тогда каждый объект x_i можно описать значениями этих характеристик — вектором $(x_{i1}, \dots, x_{in})$.

ПРИМЕР

Если оцениваем фильмы, то один объект может выглядеть таким вектором признаков: $x_i = (\text{жанр} = \text{боевик}, \text{длительность} = 120, \text{бюджет} = 150 \text{ млн рублей})$.

2. **Целевая переменная (Y)** — правильный ответ, известный для каждого объекта обучающей выборки. Это значение должна научиться предсказывать модель.

ПРИМЕР

Если есть признаки фильма, то таргетом может быть оценка фильма зрителем от 1 до 10.

x_i = (жанр, длительность, бюджет); y_i = оценка зрителя — выходное значение, которое предсказывает модель.

3. **Неизвестная целевая функция ($f : X \rightarrow Y$)** — связывает объект с правильным ответом. Модель учится приближать эту зависимость по обучающей выборке.

ПРИМЕР

В случае с фильмами f — связывает характеристики фильма с оценкой зрителя, а модель учится приближать эту зависимость. Если таргет — число (оценка от 1 до 10), то задачу можно рассмотреть как задачу регрессии, где модель прогнозирует конкретное значение рейтинга.

Дальше подробно разберём постановки регрессии, классификации и кластеризации. Ранжирование пока оставим на уровне общей карты, а к понижению размерности и детекции аномалий вернёмся позже и разберём их глубже в следующих материалах курса.

2.3. ПОСТАНОВКА ЗАДАЧИ РЕГРЕССИИ

Любой проект машинного обучения начинается с конкретного вопроса бизнеса. Компанию редко интересует сам алгоритм: ей важен прогноз, который помогает принимать решения, экономить ресурсы или увеличивать прибыль.

На этапе постановки задачи важно сформулировать, какую именно проблему надо решить, зачем это нужно бизнесу и почему машинное обучение может быть подходящим инструментом.

Рассмотрим несколько вариантов задач из разных отраслей.

- **Прогнозирование спроса (retail / supply chain)**

Вопрос бизнеса. Сколько единиц товара будет продано в следующем периоде?

Что предсказываем. Количество продаж (единицы товара за день, неделю, месяц).

Зачем. Оптимизировать запасы, сократить издержки на хранение и предотвратить дефицит товаров.

- **Оценка кредитного лимита (банкинг)**

Вопрос бизнеса. Какой кредитный лимит можно безопасно предложить клиенту?

Что предсказываем. Сумму кредитного лимита в рублях.

Зачем. Сбалансировать прибыль и риск невозврата, сделать индивидуальное предложение клиенту.

- **Прогноз времени доставки (логистика/e-commerce)**

Вопрос бизнеса. Сколько времени займёт доставка из пункта А в пункт Б?

Что предсказываем. Продолжительность доставки (в минутах или часах).

Зачем. Для повышения точности расчёта ETA, оптимизации маршрутов и управления SLA.

ОПРЕДЕЛЕНИЕ

ETA (estimated time of arrival) — ожидаемое время прибытия заказа.

SLA (service level agreement) — уровень сервиса, сроки и качества обслуживания, которые компания должна соблюдать.

- **Оценка стоимости недвижимости (real estate)**

Вопрос бизнеса. Какова справедливая рыночная цена квартиры?

Что предсказываем. Стоимость объекта в рублях.

Зачем. Для автоматической оценки активов, ценообразования и кредитного обеспечения.

В этих примерах:

- таргет будет числовой переменной (например, цена, количество, время);
- признаки должны описывать факторы, которые влияют на результат (например, количество квадратных метров влияет на стоимость квартиры);
- качество решения задачи оценивается по ошибке между предсказанным и реальным значением.



Во всех примерах результат предсказания — это действительное число, которое и должен предсказать наш будущий алгоритм. Значит, это задача регрессии.

ОПРЕДЕЛЕНИЕ

Регрессия — задача присвоения объекту произвольного числа или чисел.

ОПРЕДЕЛЕНИЕ

Одномерная регрессия — задача присвоения объекту единственного числа.

ОПРЕДЕЛЕНИЕ

Многомерная — задача присвоения объекту вектора чисел.

Регрессия относится к задаче обучения с учителем. При таком подходе модель учится на размеченных примерах. Каждое наблюдение в них описано набором признаков и имеет известный результат (таргет).

Например, чтобы оценить рыночную стоимость квартиры, нужны наблюдения с «правильными ответами» — данные по другим квартирам и их цены.

Прогнозирование цен на жильё

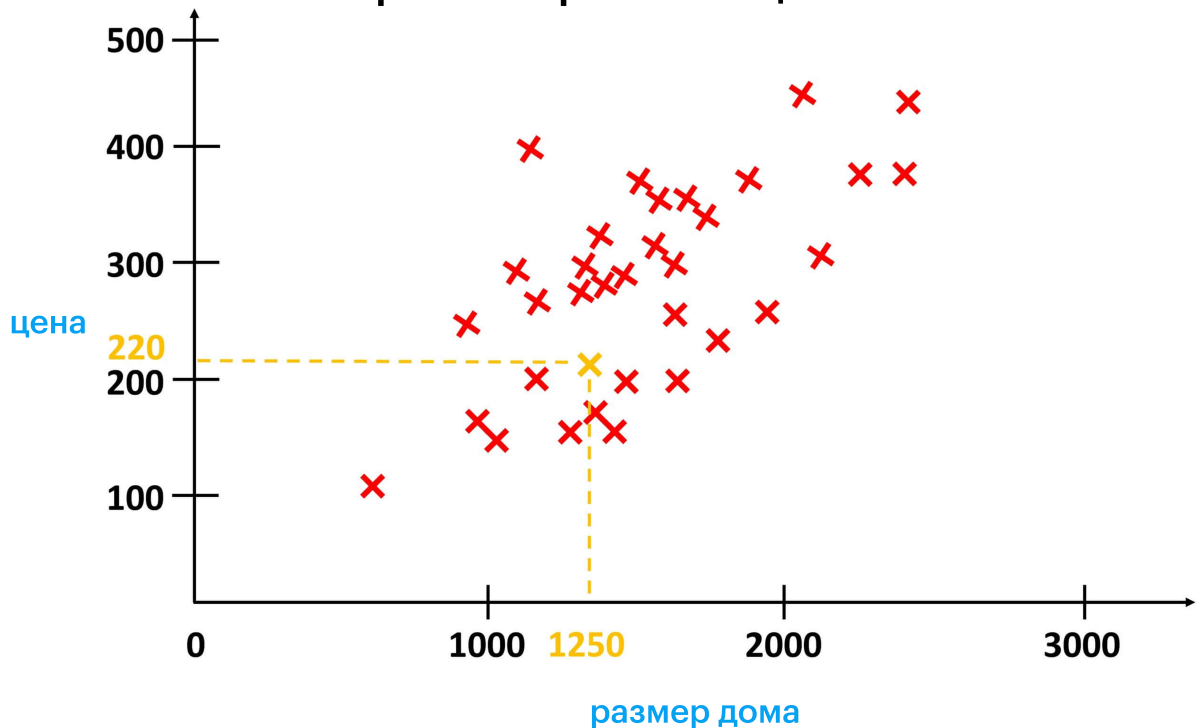


Рисунок 5. Пример графика рассеивания house size x price

2.4. ПОСТАНОВКА ЗАДАЧИ КЛАССИФИКАЦИИ

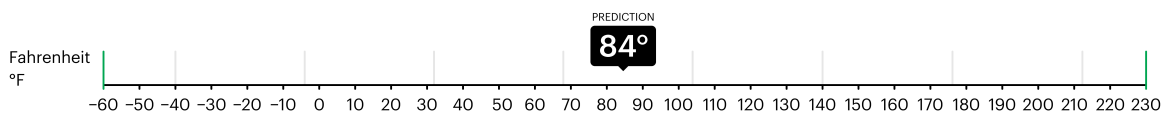
Классификация, как и регрессия — фундаментальная задача обучения с учителем в ML. Их ключевое различие заключается в типе целевой переменной, которую необходимо предсказать.

- В классификации предсказывается дискретная метка (класс) из ограниченного множества.
- В регрессии предсказывается непрерывное численное значение.



Regression

What is the temprature going to be tomorrow?



Classification

Will it be Cold or Hot tomorrow?

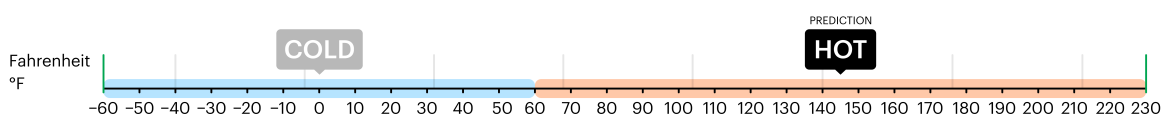


Рисунок 6. Разница между регрессией и классификацией

Таким образом:

ОПРЕДЕЛЕНИЕ

классификация — это задача предсказания категории или класса из конечного множества допустимых меток.

Модель учится по признакам определять, к какому классу относится новый объект.

Рассмотрим несколько вариантов задач из разных отраслей.

- **Обращения клиентов (customer support)**

Вопрос бизнеса. К какому типу относится обращение клиентов?

Что предсказываем. Категория: доставка, оплата, возврат, техническая проблема.

Зачем. Сократить время обработки и автоматически направлять обращения нужным специалистам.

- **Одобрение кредита (банкинг)**

Вопрос бизнеса. Следует ли одобрить заявку на кредит?

Что предсказываем. Класс: одобрить/отказать.

Зачем. Снизить риск невозврата кредита и автоматизировать принятие решения.

По числу классов и количеству меток у одного объекта выделяют несколько видов классификации.

Дальше рассмотрим виды задач классификации.

Бинарная классификация

ОПРЕДЕЛЕНИЕ

Бинарная классификация (англ. binary classification) — это самый базовый случай задачи классификации, в котором существует только два класса.

Например, «да» или «нет», «болен» или «здоров», «положительный» или «отрицательный».

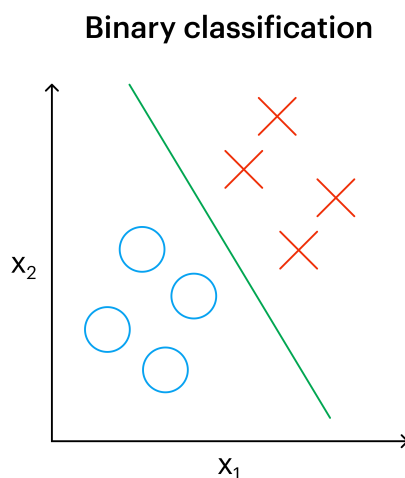


Рисунок 7. Бинарная классификация

Многоклассовая классификация

ОПРЕДЕЛЕНИЕ

Многоклассовая классификация (англ. multiclass classification) — задача классификации, в которой модель учится предсказывать один из нескольких (больше двух) взаимоисключающих классов. При этом каждый объект выборки принадлежит только одному классу из множества.

Multi-class classification

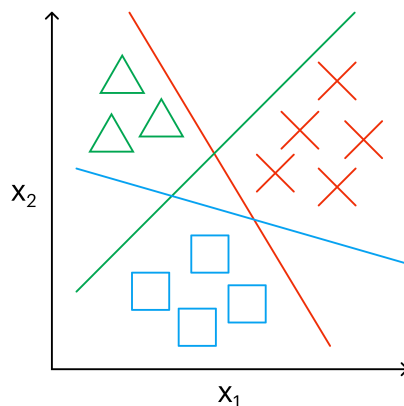


Рисунок 8. Многоклассовая классификация

Многометочная классификация

ОПРЕДЕЛЕНИЕ

Многометочная классификация (англ. multilabel classification) — задача классификации, в которой каждому объекту могут соответствовать несколько классов одновременно и они не взаимоисключающие.

Например, под такую классификацию попадают теги товара в рекомендательной системе. Подобную задачу можно рассмотреть как набор независимых бинарных классификаций — по одному для каждой возможной метки.

Multilabel classification

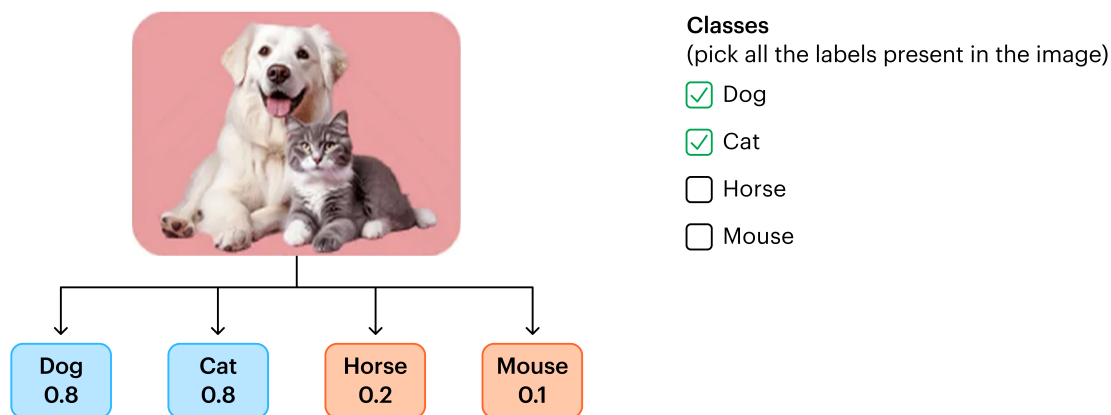


Рисунок 9. Многометочная классификация

2.5. ПОСТАНОВКА ЗАДАЧИ КЛАСТЕРИЗАЦИИ

Во многих практических задачах приходится работать с большим объёмом данных, для которых отсутствует какая-либо разметка. При этом нет информации о метке класса, как в задаче классификации, количество групп может задаваться заранее или определяться в процессе решения задачи.

Для таких сценариев существует кластеризация. Её цель — на основе внутренней структуры данных разделить объекты на группы — кластеры — так, чтобы объекты внутри одного кластера были максимально похожи, а объекты разных кластеров — максимально различны.

Рассмотрим пару типичных ситуаций из разных отраслей.

- **Сегментация клиентов (retail/e-commerce)**

Вопрос бизнеса. Какие типы клиентов существуют в нашей базе?

Что ищем. Группы клиентов с похожим поведением (частота покупок, чек, категории, реакции на скидки).

Зачем. Для персонализации предложений, маркетинговых кампаний и настройки разных стратегий взаимодействия.

- **Выявление типов транзакций (fintech)**

Вопрос бизнеса. Какие типы операций встречаются в данных?

Что ищем. Группы транзакций с похожими характеристиками (суммы, частота, время, география).

Зачем. Для антифрод-аналитики, мониторинга аномалий и дополнительной проверки нетипичных операций.

Во всех этих примерах:

- таргета в классическом смысле нет — алгоритму не сообщают «правильные» ответы;
- объекты описываются набором признаков, отражающих их свойства или поведение;
- результат — это разбиение объектов на группы, а не предсказание числа или класса;
- качество решения оценивается через устойчивость, интерпретируемость и практическую полезность кластеров, а не через ошибку предсказания.

Значит, это задача кластеризации.

ОПРЕДЕЛЕНИЕ

Кластеризация — задача автоматического разбиения объектов на группы на основе их сходства без использования целевой переменной.

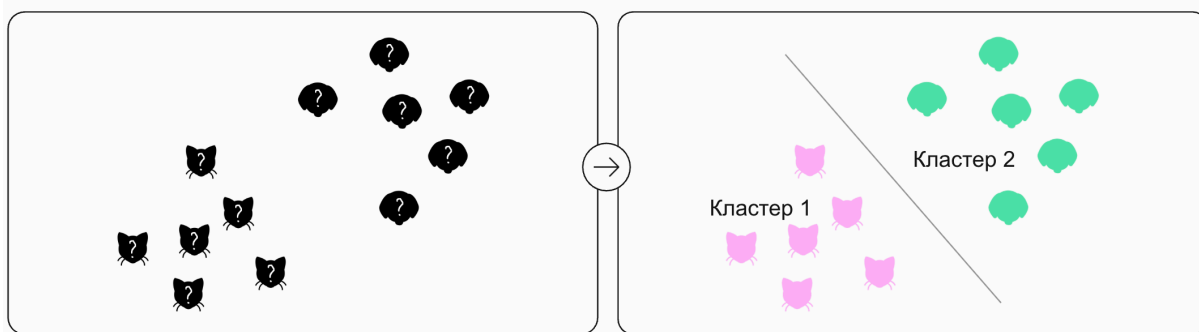


Рисунок 10. Визуализация задачи кластеризации

Кластеризация относится к задачам **обучения без учителя**. Алгоритм обучается только на признаках объектов и сам выявляет структуру данных. Дополнительные бизнес-показатели (например, churn, LTV, конверсия) могут использоваться после — для анализа и интерпретации полученных кластеров, но не участвуют в обучении.

3. Если кратко

- Работа над ML-проектом проходит несколько связанных этапов: от постановки задачи и подготовки данных до обучения, оценки, запуска и поддержки решения. Этот процесс итеративный — при необходимости команда возвращается к предыдущим этапам.
- CRISP-DM даёт классическую схему работы над ML-проектом. В курсе мы используем более подробный ML Lifecycle из семи этапов.
- В обучении с учителем алгоритм учится по объектам с известными ответами. В обучении без учителя известных ответов нет — алгоритм ищет структуру в самих данных.
- Множество объектов X есть в любой ML-задаче. Множество ответов Y и целевая зависимость $y : X \rightarrow Y$ появляются в задачах обучения с учителем.
- В регрессии модель предсказывает число, в классификации — один или несколько классов, а в кластеризации разбивает объекты на группы без известных правильных ответов.
- Выбранная постановка задачи определяет формат результата, подходящие модели и способ оценки качества.